Digital Programming (EL467)

Course Project: WHACK-A-MOLE game implementation on FPGA

Team members:

1. Solanki Jaitra Jitendrasinh (202304005)
2. Patel Parth Kamleshbhai (202304008)

Project Background / Overview:

The project **"Whack-a-Mole Game Implementation on FPGA"** is a digital hardware version of the popular arcade game *Whack-a-Mole*. The main goal of this project is to design an interactive game that runs completely on an **FPGA board (Nexys A7)** without using any external microcontroller or software. The game uses LEDs, switches, and a 7-segment display to create a fun and engaging experience while demonstrating important digital design concepts.

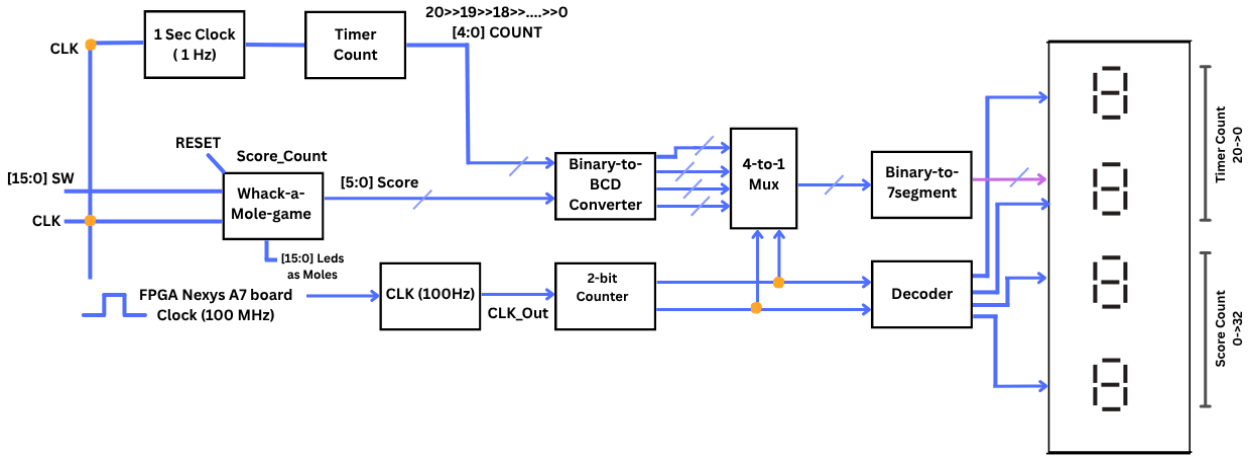
In this game, different LEDs represent moles appearing randomly. The player's task is to quickly press the corresponding switch when an LED lights up. Every correct press increases the player's score, while incorrect or missed attempts are ignored. The game runs for a fixed duration of **20 seconds**, and after that, the final score is displayed on the 7-segment display.

The FPGA board's **100 MHz clock** is divided into slower clocks to control different parts of the game. A **1 Hz clock** is used for the timer, counting down from 20 to 0 seconds, and a **100 Hz clock** is used to refresh the 7-segment display. The main **Whack-a-Mole game module** manages the logic for LED activation, user inputs from switches, and score updates.

The **Binary-to-BCD converter**, **4-to-1 multiplexer**, and **Binary-to-7-segment decoder** work together to show both the timer and score on the 4-digit 7-segment display. The top two digits show the remaining time, while the bottom two digits show the player's score, which can go up to 32.

This project helped in understanding how to design real-time systems using Verilog HDL and FPGA-based digital circuits. It combines concepts like counters, multiplexers, clock dividers**,** and state machines into one practical and enjoyable project. Overall, this game demonstrates how digital logic and timing can be applied creatively to make an interactive hardware-based system.

Block/circuit diagrams:



The block diagram shows how different modules like the game logic, counters, and display units are connected. It explains the flow of signals for timing, score calculation, and display control in the Whack-a-Mole game.

- Now a detailed explanation of all these Modules is given below.

1) Module Name: `timerclock` (1 Hz Clock Generator)

Purpose:

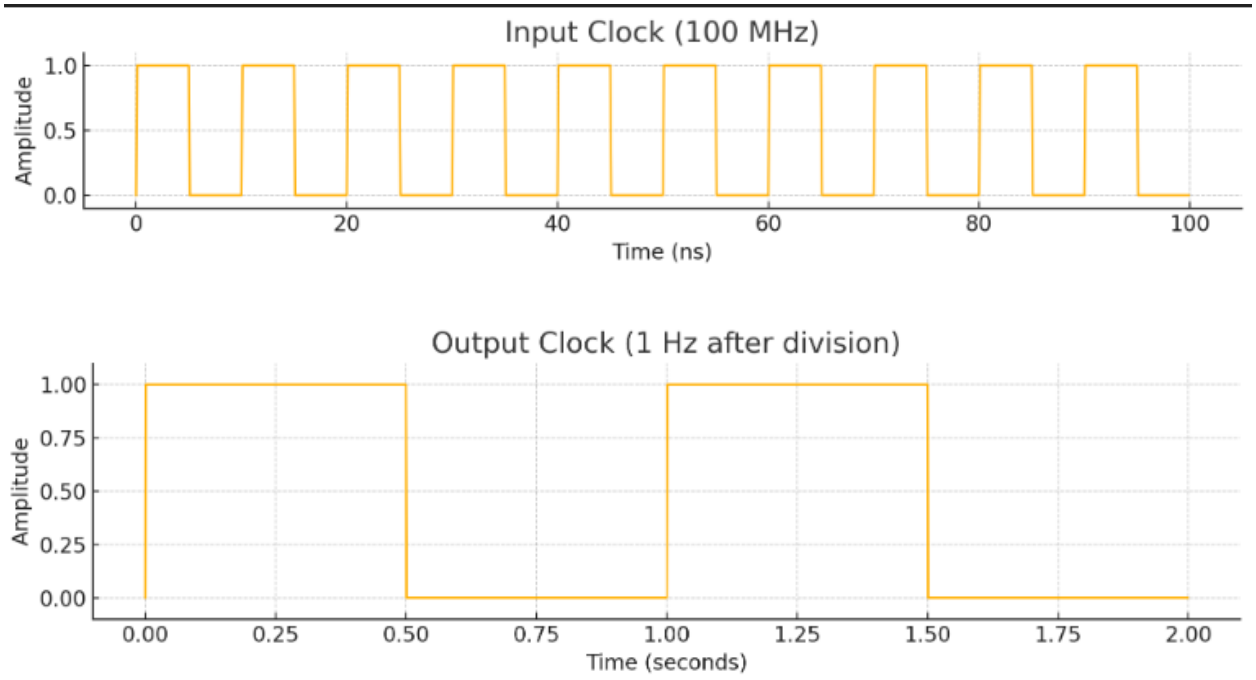
This module is used to convert the 100 MHz clock from the FPGA into a 1 Hz clock. In simple terms, it slows down the clock signal so that it can be used for time-based operations like counting seconds in the game timer.

How the Code Works:

1. Input and Output:
    - `clk_in` → the 100 MHz main clock from the FPGA board.
    - `clk_out` → the 1 Hz clock output (1 pulse every second).
2. Counter Declaration:
    - A 28-bit register `period_count` is declared to store the number of clock pulses counted.
    - Since the input clock is 100 MHz, one second equals 100,000,000 pulses.
3. Clock Division Logic:
    - On every positive edge of the input clock, the counter increases by 1.

- When the counter reaches 100,000,000 - 1, it resets to 0.
- At that moment, the output `clk_out` is set high for one clock cycle — producing a 1 Hz pulse.

4. Output Behavior:
   - The output clock (`clk_out`) becomes high once per second, effectively dividing the 100 MHz clock by 100 million.



2) Module Name: `timercount` (20-Second Countdown Timer)

Purpose:

The `timercount` module is used to **generate a 20-second countdown timer** for the Whack-a-Mole game. It helps control the total playtime of the game. The timer starts from 20 seconds and decreases by one every second until it reaches zero.

Logical Working of the Code:

1. Clock Source:

- The module takes the main FPGA clock **(`clk`)** and passes it through the `timer clock` module to get a 1 Hz clock signal **(`clk_out`)**.
- This 1 Hz clock ensures that the counter decreases exactly once per second.

2. Counter Initialization:
   - A 5-bit register, `current_count,` is initialized with the value **20**, representing the total game duration in seconds.
3. Counting Logic:
   - The counter decreases by 1 on every positive edge of the 1 Hz clock.
   - If the reset button is pressed, the timer resets to 20.
   - When the counter reaches 0, it stops counting and holds the value at 0.
4. Output:
   - The counter's current value is continuously assigned to the output port `count`, which can be displayed on the 7-segment display.

3) Module Name: `whack_a_mole_game` (Main Game Logic)

Purpose:

The `whack_a_mole_game` module controls the overall game logic of the Whack-a-Mole project. It manages the LED lights (representing moles), monitors player input from switches, updates the score, and transitions between different game states. This is the main control unit that decides which LED will light up next and when the score should increase.

Logical Working of the Code:

1. Inputs and Outputs:
   - Inputs:
     - `clk`: Clock signal to synchronize game logic.
     - `sw[15:0]`: Represents the 16 switches.
     - `reset`: Used to restart the game.
   - Outputs:

        - `score_count`: Stores the player's score (maximum 32 points).
        - `led[15:0]`: Controls the 16 LEDs (each LED represents a mole).

2. State Machine Design:
    - The game uses a Finite State Machine (FSM) approach with 33 defined states (`s000000` to `s100000`).
    - Each state corresponds to a specific mole (LED) lighting up and waiting for the player to hit the correct switch.
    - When the correct switch is pressed, the FSM moves to the next state, the score increases by 1, and a new LED lights up.
3. Reset and State Transition:
    - When the `reset` is pressed, the game returns to the initial state (`s000000`), and the score resets to 0.
    - If no switch is pressed, the game remains in the same state.
    - The FSM ensures that the LEDs light up one by one, following a predefined pattern.
4. Scoring System:
    - Every time the player hits the correct switch (matching the glowing LED), the score (`score_count`) increases by 1.
    - The maximum achievable score is 32 points, after which the game remains in the final state.

Table: <u>Example of Game State Transitions</u>

Current State	LED ON (Mole Position)	Switch Pressed (Correct Input)	Next State	Score Count	Description
s000000	LED[3]	SW[3]	s000001	1	Game starts — first mole appears at position 3.

s000001	LED[7]	SW[7]	s000010	2	The player hits the correct switch; next mole lights at position 7.
s000010	LED[12]	SW[12]	s000011	3	Next mole lights at position 12.
s000011	LED[0]	SW[0]	s000100	4	The player responds correctly again.
s000100	LED[5]	SW[5]	s000101	5	Score increases; a new mole appears at position 5.
s000101	LED[14]	SW[14]	s000110	6	The game continues with the mole at position 14.
...	...	...	...	...	The sequence continues similarly until the final state.
s100000	None	—	s100000	32	The maximum score reached in the game ends.

4) Module Name: `Slow_Clock` (100 Hz Clock Divider)

Purpose:

The `Slow_Clock` module is used to convert the 100 MHz clock from the FPGA into a much slower 100 Hz clock signal.This slower clock is mainly used to control display refreshing and LED blinking, where high-speed toggling would be invisible to the human eye.

Logical Working of the Code:

1. Input and Output:
    - `clk_in` → 100 MHz main clock from the FPGA board.
    - `clk_out` → Output clock signal with 100 Hz frequency.
2. Counter Operation:
    - A 21-bit register **(`count`)** is used to keep track of the number of input clock pulses.
    - On every rising edge of `clk_in`, the counter increases by 1.
3. Clock Division Logic:
    - When the counter reaches 50,000, it resets back to 0 and toggles **(`~clk_out`)** the output clock.
    - Because `clk_out` toggles every 50,000 counts, a full cycle takes 100,000 counts, dividing 100 MHz by 100,000 = 1 kHz.
    - Each toggle gives half the period, resulting in a 100 Hz clock output.
4. Output Behavior:
    - The output clock alternates between 0 and 1 one hundred times per second, producing a clean square-wave signal.
    - This frequency is suitable for refreshing 7-segment displays and controlling LED blinking without flicker.

5) Module Name: `BCD_Binary_Converter` (Binary to BCD Conversion using Shift-Add-3 Algorithm)

Purpose:

The `BCD_Binary_Converter` module converts an 8-bit binary number (such as a score or time value) into its BCD (Binary-Coded Decimal) form. This conversion is essential because 7-segment displays can only display decimal digits (0–9), not raw binary numbers.

Logical Working of the Code:

1. Overview:

- The module uses the Shift-Add-3 algorithm, a standard method to convert binary numbers into decimal form.
- It divides the binary number into hundreds, tens, and ones digits for easy display.

2. How It Works (Step-by-Step):
   - The 8-bit binary input (`A`) is processed bit by bit using a series of Shift-Add-3 operations.
   - For each shift:
     - If a 4-bit group is greater than or equal to 5, the algorithm adds **3** to it (this ensures proper decimal alignment).
     - Otherwise, it just shifts the bits to the left.
   - This process continues until all bits are shifted and the final BCD digits are obtained.
3. Module Structure:
   - The design consists of two modules:
     - `Shift_Add3_algorithm`: Performs the Add-3 correction for each 4-bit segment.
     - `BCD_Binary_Converter`: Connects multiple `Shift_Add3_algorithm` blocks to form the full conversion path.
   - Intermediate wires (`c1–c7`, `d1–d7`) carry partial results between the blocks.
4. Outputs:
   - **`ones`** → Last 4 bits represent the unit place.
   - **`tens`** → Next 4 bits represent the tens place.
   - **`hundreds`** → 2 bits represent the hundreds place.

6) Module Name: `two_bit_counter` (2-Bit Sequential Counter)

Purpose:

The `two-bit_counter` module generates a 2-bit counting signal that cycles through values `00`, `01`, `10`, and `11`. It is typically used to control the multiplexing of the 7-segment display, where each digit is displayed one at a time in rapid succession, giving the illusion that all digits are ON simultaneously.

Logical Working of the Code:

1. Input and Output:
    - `clk` → A slow clock input (usually 100 Hz) that determines how fast the counter changes.
    - `Q[1:0]` → The 2-bit output of the counter, representing four possible states.
2. Counter Operation:
    - On every positive edge of the clock, the counter increments by 1.
    - The 2-bit output wraps around automatically after reaching `11` (3 in decimal), returning to `00`.

This creates a repeating sequence:

```
00 → 01 → 10 → 11 → 00 → ...
```

3. Initialization:
    - The counter is initialized to `0` at the start, ensuring a defined starting point when the circuit powers up.
4. Usage in the Project:
    - The 2-bit output (`Q`) can be used as a select signal for a 4-to-1 multiplexer that chooses which 7-segment display digit (ones, tens, hundreds, etc.) to show at a given instant.
    - Because the switching happens very fast (100 times per second), the human eye sees all digits glowing continuously.

7) Module Name: `four_one_mux` (4-to-1 Multiplexer)

Purpose:

The `four_one_mux` module selects one out of four 4-bit inputs (`A`, `B`, `C`, `D`) based on a 2-bit select signal (`SS`) and passes it to the output `Y`. In the *Whack-a-Mole* project, this module is used to multiplex the BCD digits (ones, tens, hundreds, etc.) so that multiple digits can share the same 7-segment display hardware.

Logical Working of the Code:

1. Inputs and Outputs:
    - Inputs:
        - `A`, `B`, `C`, `D`: Four 4-bit data inputs (for example, BCD values for different digits).
        - `SS[1:0]`: 2-bit select line that decides which input is passed to the output.
    - Output:
        - `Y[3:0]`: The selected 4-bit output based on `SS`.
2. Selection Logic:
    - When `SS = 00`, output `Y` = input `A`.
    - When `SS = 01`, output `Y` = input `B`.
    - When `SS = 10`, output `Y` = input `C`.
    - When `SS = 11`, output `Y` = input `D`.
    - If none of these match (default case), output `Y` = `0000`.
3. Usage in the Project:
    - The multiplexer works together with the 2-bit counter **(`two_bit_counter`)**.
    - The counter generates a select signal (`SS`) that changes continuously (00 → 01 → 10 → 11).
    - Each select value allows one digit (ones, tens, hundreds, or score) to be displayed briefly.
    - This happens so quickly that all digits appear simultaneously due to persistence of vision.

8) Module Name: `BCD_SevenSegment` (BCD to 7-Segment Display Decoder)

Purpose:

The `BCD_SevenSegment` module converts a 4-bit BCD (Binary-Coded Decimal) input into a 7-bit output pattern that drives a 7-segment display. Each bit in the output corresponds to one segment of the display (a–g), allowing the display to form digits from 0 to 9. This module is responsible for showing the timer countdown and score values clearly on the FPGA board.

Logical Working of the Code:

1. Inputs and Outputs:
    - Input (**`Y[3:0]`**)**:** A 4-bit BCD value (ranging from 0000 to 1001).
    - Output (**`disp[6:0]`**)**:** A 7-bit signal that controls the seven segments of the display.
        - Each bit corresponds to a display segment: **a, b, c, d, e, f, g**.
        - A `0` usually turns a segment ON (active-low logic on most FPGA boards).
2. Case Statement Function**:**
    - The `case` block checks the input `Y` and assigns the corresponding 7-segment pattern.
    - For example:
        - `Y = 0000` → Displays **0** (all segments ON except g).
        - `Y = 0001` → Displays **1** (segments b and c ON).
        - `Y = 0010` → Displays **2** (segments a, b, g, e, d ON).
    - The pattern continues up to `Y = 1001` for digit **9**.
    - If any invalid BCD value appears, the `default` case ensures the display stays blank or shows zero.
3. Working Principle:
    - When a binary value (from the counter or BCD converter) is provided, this module automatically lights up the correct combination of LEDs on the 7-segment display to show the corresponding decimal digit.

9) Module Name: `decoder2to8` (2-to-8 Display Decoder)

Purpose:

The `decoder2to8` module is designed to activate one of the 8 seven-segment displays at a time, based on a 2-bit select input (`en`).In the Whack-a-Mole game, this module helps in time-multiplexing the 7-segment display, allowing multiple digits (like timer and score) to appear active simultaneously, even though only one display is actually ON at a time.

Logical Working of the Code:

1. Inputs and Outputs:
    - Input **(`en[1:0]`):** A 2-bit signal from the counter or multiplexer that decides which display to activate.
    - Output **(`an[7:0]`):** Controls the enable pins of all eight 7-segment displays (active-low).
        - `0` = Display ON
        - `1` = Display OFF
2. Case Logic:
    - The module uses a `case` statement to assign a unique 8-bit pattern to the output depending on `en`.
    - Example operation:
        - `en = 00` → Rightmost display ON (`1111_1110`)
        - `en = 01` → 2nd display ON (`1111_1101`)
        - `en = 10` → 3rd display ON (`1111_1011`)
        - `en = 11` → 4th display ON (`1111_0111`)
    - When none of the valid inputs are active, all displays remain OFF (`1111_1111`).
3. Working Principle:
    - The decoder works together with the 2-bit counter (`two_bit_counter`).
    - The counter cycles through `00 → 01 → 10 → 11`, activating each display one by one.
    - Since this happens very fast (hundreds of times per second), the human eye perceives that all four digits are glowing continuously.

- This module acts like a selector switch that decides which 7-segment display should be ON at a time. It quickly switches between displays so that all digits (timer and score) appear visible at once to the human eye.

10) Module Name: `main_whack_a_mole_top` (Top-Level Integration Module)

Purpose:

The `main_whack_a_mole_top` module acts as the central controller of the Whack-a-Mole game. It integrates all the functional modules — including the timer, game logic, binary-to-BCD converters, multiplexers, decoders, and display drivers — into one complete system that runs on the FPGA board. This module coordinates the interaction between input switches, LEDs, the timer countdown, and the 7-segment display, making the game playable and visually responsive.

Logical Working of the Code:

1. Clock and Reset Inputs:
    - The FPGA's 100 MHz clock is provided as `clk`.
    - A push-button input `reset` resets both the game and the timer.
    - A slow clock (`clk_100hz`) is generated using the `Slow_Clock` module for visible LED and display updates.
2. Game Logic (Whack-a-Mole):
    - The `whack-a-mole game` module handles the core gameplay.
    - It lights up one LED at a time (the "mole"), reads the player's switch inputs, and updates the score counter accordingly.
3. Timer Control:
    - The `timercount` module generates a 20-second countdown, showing how much time the player has left.
    - The timer decreases by one every second using the 1 Hz divided clock.
4. Binary-to-BCD Conversion:
    - Both the **timer** and **score** values (in binary) are converted into BCD format using two `BCD_Binary_Converter` modules.
    - This conversion allows the values to be shown correctly on the 7-segment displays.
5. Multiplexing and Display:

- A 2-bit counter (**`two_bit_counter`**) continuously cycles through four states (`00 → 01 → 10 → 11`), selecting one display digit at a time.
- The 4-to-1 multiplexer (**`four_one_mux`**) chooses which BCD digit (timer or score) to show.
- The decoder (**`decoder2to8`**) activates the corresponding display (active-low).
- The **`BCD_SevenSegment`** module then converts the 4-bit BCD digit into the correct 7-segment LED pattern.
- Together, these create a time-multiplexed display, where all digits appear to glow simultaneously.

6. LED Outputs:
    - The 16 LEDs on the board represent mole positions.
    - When a mole “pops up,” the corresponding LED glows, and the player must press the matching switch to score points.

Conclusion

The Whack-a-Mole game was successfully designed and implemented on the FPGA using Verilog. The system combines clock division, a countdown timer, game logic, BCD conversion, multiplexing, and 7-segment display control to create a fully working hardware-based game. The LEDs act as moles, the switches are used as inputs, and the score and timer are displayed correctly on the 7-segment display.

This project helped in understanding practical concepts like FSM design, sequential logic, clock management, and display interfacing. Overall, the game works smoothly in real time and demonstrates how digital logic can be used to build an interactive system on an FPGA.

Future Scope

- Add difficulty levels.
- Add sound/buzzer feedback.
- Improve display with VGA graphics.